

DroneMapper

High Level Design Document

Roni Hagai (324207257) | Noy Elyasi (315659201)

1. Overview

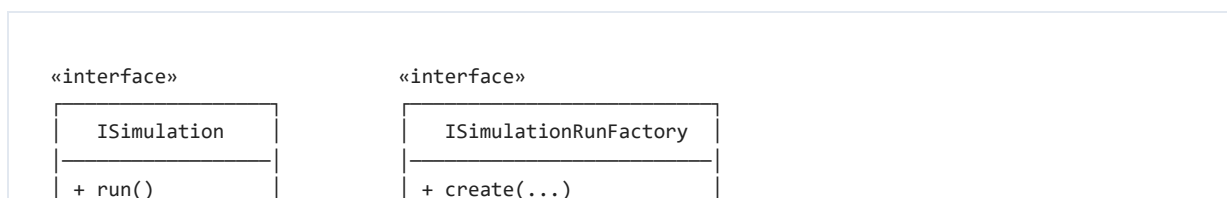
DroneMapper is an autonomous 3D indoor-mapping simulation written in C++20. A virtual drone navigates a user-supplied voxel building model loaded from a .npy file, fires a cone-beam Lidar sensor at each position, and constructs an internal occupancy map written to disk when exploration finishes. A separate `maps_comparison` tool scores the drone's reconstructed map against the ground-truth using an F1-based metric. The system supports batch simulation runs driven by a single YAML composition file.

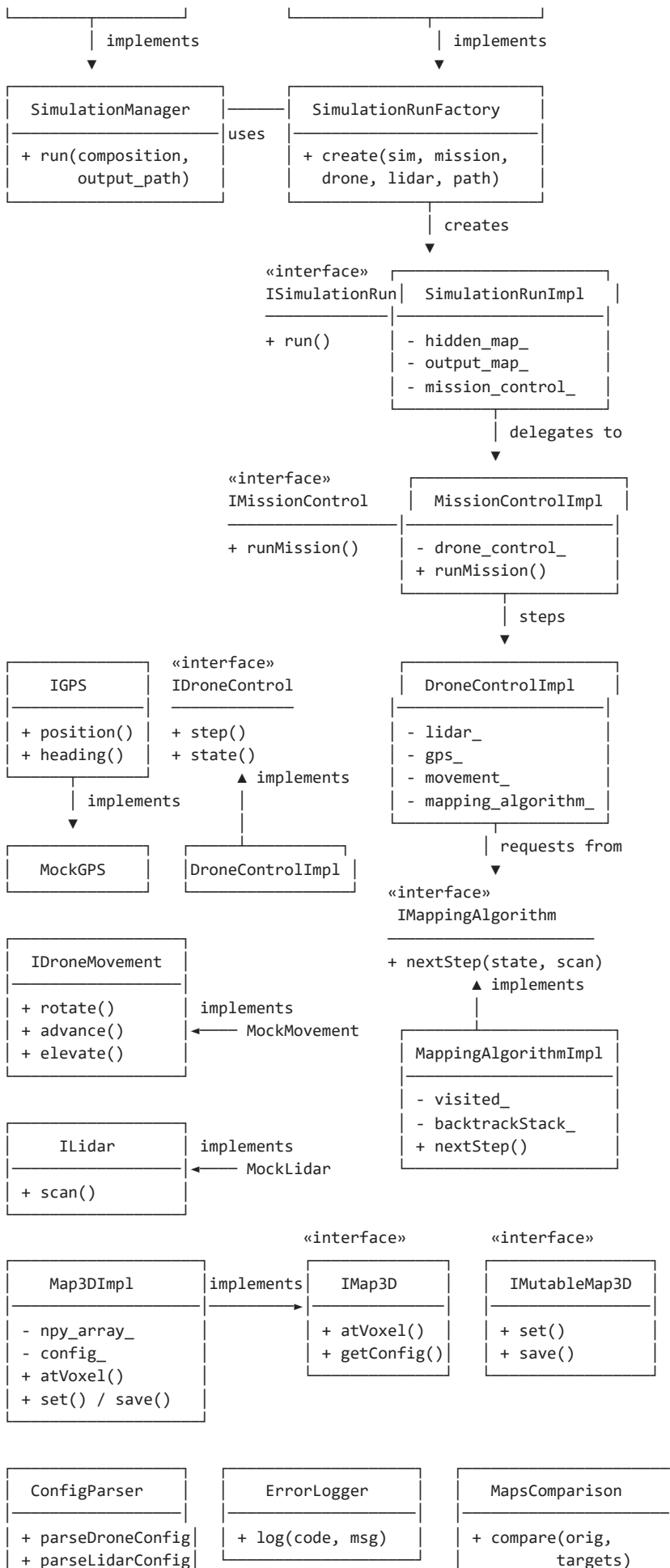
2. High-Level Architecture

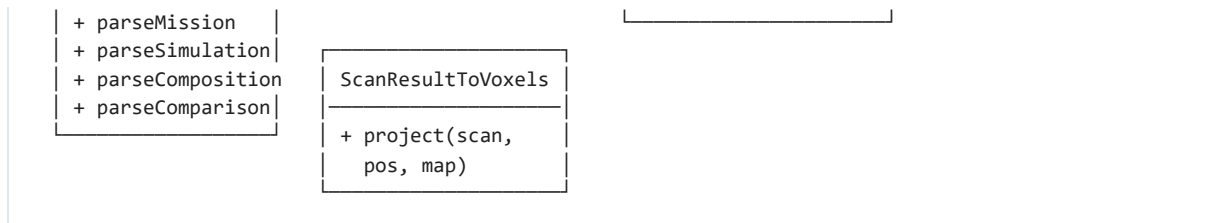
The system is divided into six layers. Each layer communicates only with its immediate neighbours through well-defined interfaces, enabling independent testing of every component.

Layer	Component	Responsibility
Entry Points	<code>drone_mapper_simulation_main.cpp</code> <code>maps_comparison_main.cpp</code>	Parse CLI args; delegate to <code>SimulationManager</code> or <code>MapsComparison</code>
Orchestration	<code>SimulationManager</code> / <code>ISimulation</code> <code>SimulationRunFactory</code> / <code>ISimulationRunFactory</code>	Combinatorial run scheduling; factory wires all dependencies for a single run
Run Execution	<code>SimulationRunImpl</code> / <code>ISimulationRun</code> <code>MissionControlImpl</code> / <code>IMissionControl</code>	Owns one simulation run; drives drone steps; manages termination & output
Drone Control	<code>DroneControlImpl</code> / <code>IDroneControl</code> <code>MappingAlgorithmImpl</code> / <code>IMappingAlgorithm</code>	Executes per-step commands from the algorithm; DFS exploration logic
Sensor Mocks	<code>MockGPS</code> / <code>IGPS</code> <code>MockMovement</code> / <code>IDroneMovement</code> <code>MockLidar</code> / <code>ILidar</code>	Simulate physical hardware against the hidden ground-truth map
Data / Config	<code>Map3DImpl</code> / <code>IMap3D</code> / <code>IMutableMap3D</code> <code>ConfigParser</code> · <code>ErrorLogger</code> · <code>MapsComparison</code> · <code>ScanResultToVoxels</code>	3D occupancy grid (.npy), YAML parsing, error logging, scoring, scan projection

3. UML Class Diagram



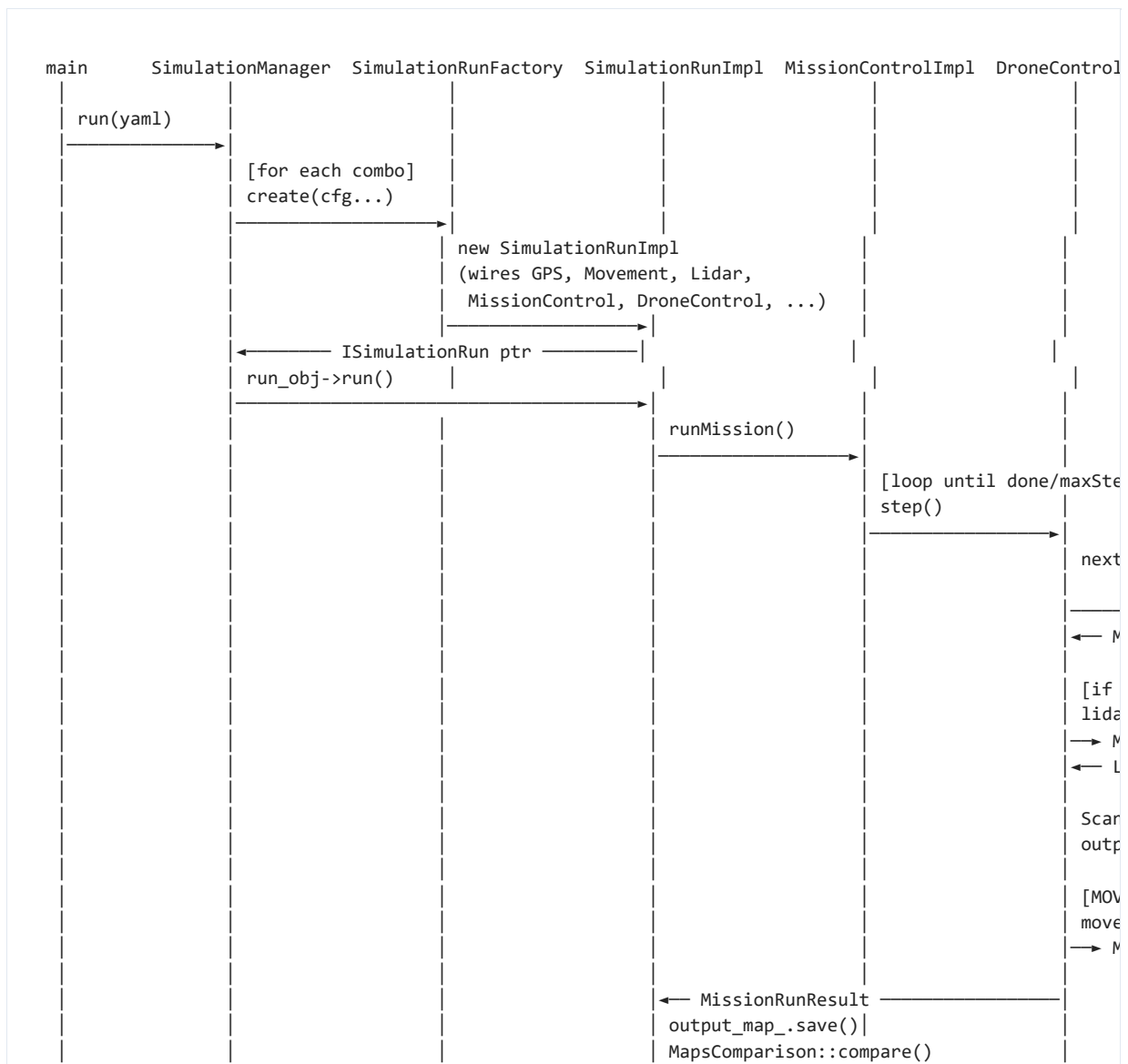




Key relationships:

- **DroneControlImpl** → **Interfaces**: depends on `IGPS`, `IDroneMovement`, `ILidar`, `IMappingAlgorithm`, `IMutableMap3D` — never on concrete implementations (Dependency Inversion Principle).
- **MappingAlgorithmImpl** → **IMap3D**: the algorithm reads the output map (read-only) to guide DFS; it never writes.
- **SimulationRunFactory**: the composition root — creates all objects and injects dependencies for a single run.
- **Map3DImpl**: implements both `IMap3D` (read) and `IMutableMap3D` (write); factory passes the same instance under the appropriate interface to each consumer.

4. UML Sequence Diagram — Main Flow





5. Component Descriptions

5.1 Entry Points

drone_mapper_simulation_main.cpp

Accepts a single CLI argument: path to a YAML composition file. Parses it via `ConfigParser::parseCompositionConfig()`, constructs `SimulationManager` with a `SimulationRunFactory`, calls `run()`, and writes `simulation_output.yaml` summarising all run scores.

maps_comparison_main.cpp

Accepts two or more `.npy` file paths (ground-truth first, then one or more drone maps). Loads each as a `Map3DImpl`, calls `MapsComparison::compare()`, and prints one integer score (0–100) per target map.

5.2 SimulationManager — Combinatorial Orchestrator

Implements `ISimulation`. Receives a `SimulationCompositionData` containing lists of simulation configs, mission configs, drone configs, and lidar configs. Iterates over the Cartesian product (sim × mission × drone × lidar), asking the injected `ISimulationRunFactory` to create an `ISimulationRun` for each combination, calls `run()` on it, and accumulates results into a `SimulationManagerReport`. Factory exceptions are caught and recorded as error runs without aborting the remaining combinations.

5.3 SimulationRunFactory — Composition Root

Implements `ISimulationRunFactory`. For each combination it:

- Parses all four YAML config files via `ConfigParser`.
- Loads the ground-truth `.npy` map into a `Map3DImpl` (hidden map, read-only).
- Creates an empty output `Map3DImpl` (mutable, same resolution/offset).
- Constructs `MockGPS`, `MockMovement`, `MockLidar` — each referencing the hidden map.
- Constructs `MappingAlgorithmImpl` (receives drone config + read-only output map ref).
- Constructs `DroneControlImpl` (receives all sensor interfaces + algorithm + mutable map).
- Constructs `MissionControlImpl` (receives drone control + mission config).
- Returns a fully-wired `SimulationRunImpl`.

5.4 SimulationRunImpl — Single-Run Driver

Implements `ISimulationRun`. Its `run()` method:

- Calls `MissionControlImpl::runMission()` and receives a `MissionRunResult`.

- Saves the output map to a timestamped `.npy` file under `output_results/`.
- Computes the mapping score via `MapsComparison::compare(hidden_map, output_map)`.
- Returns a `SimulationResult` containing all config data, status, file path, and score.

5.5 MissionControlImpl — Step Loop

Implements `IMissionControl`. Calls `IDroneControl::step()` in a loop until the drone reports `Completed` or `FinishedWithUnmappable`, or until `max_steps` is reached. Collects any error references and returns a `MissionRunResult` with final status and step count.

5.6 DroneControlImpl — Per-Step Executor

Implements `IDroneControl`. On each `step()` call:

1. Reads current pose from `IGPS`.
2. Passes pose + last scan result to `IMappingAlgorithm::nextStep()`.
3. Executes the returned `MappingStepCommand::scan` via `ILidar`, project hits into `IMutableMap3D` via `ScanResultToVoxels`, or move via `IDroneMovement`.
4. Returns `DroneStepResult` (`Continue` / `Completed` / `Error`).

5.7 MappingAlgorithmImpl — DFS Exploration

Implements `IMappingAlgorithm`. Runs a deterministic depth-first search over the 3D voxel grid. Reads the output map through the read-only `IMap3D` interface to check occupancy of candidate cells.

State

- `visited_` — `std::unordered_set<GridPoint>`: prevents revisiting cells.
- `backtrackStack_` — `std::stack<GridPoint>`: DFS return path.
- `phase_` — navigation phase state machine (`FindNext` / `Elevating` / `Rotating` / `Advancing` / `Done`).

nextStep() — algorithm tick

- **Scan phase:** If scan result is available, call `ScanResultToVoxels` to project hits into the output map.
- **FindNext:** Select an unvisited, unoccupied, in-bounds neighbour (`+X,-X,+Y,-Y,+Z,-Z`); push current cell; transition to `Elevating`.
- **Elevating / Rotating / Advancing:** Emit movement commands one sub-step at a time, checking for collision responses.
- **Backtrack:** If no frontier, pop the stack. Return `Finished` when the stack is empty.

5.8 Abstract Interfaces

`DroneControlImpl` depends on five pure abstract classes, enabling full dependency inversion (DIP):

- `IGPS` — `position()` → `Position3D`, `heading()` → `Orientation`.
- `IDroneMovement` — `rotate(direction, angle)`, `advance(distance)`, `elevate(distance)` → `MovementResult`.

- `ILidar` — `scan(orientation) → LidarScanResult; config() → LidarConfigData`.
- `IMappingAlgorithm` — `nextStep(state, scan*) → MappingStepCommand`.
- `IMap3D` — `atVoxel(pos), getMapConfig(), isInBounds(pos)`.
- `IMutableMap3D` (extends `IMap3D`) — `set(pos, occupancy), save(path)`.

5.9 Mock Implementations

MockGPS

Stores the drone's exact world-space position (`Position3D`) and heading (`Orientation`). Both `MockMovement` and `MockLidar` hold a reference to the same `MockGPS` instance — the movement mock writes to it, the lidar mock reads from it.

MockMovement

Applies rotate/advance/elevate commands to the `MockGPS` state. Before every translational move it ray-marches the drone's bounding box through the hidden ground-truth map and rejects the move (returning `MovementResult::Blocked`) if any sample is Occupied or outside mission bounds.

MockLidar

Computes all beam directions for the configured FOVC cone (circle k has 4^k beams, half-angle = $\text{atan}(k \cdot d / z_{\min})$). Each beam is ray-marched through the hidden map. Returns:

- $0 * \text{cm}$ — hit closer than $z_{\min}$.
- $t * \text{cm}$ — distance to first occupied voxel ($z_{\min} \leq t \leq z_{\max}$).
- $z_{\max} * \text{cm}$ — no obstacle within range.

5.10 Map3DImpl — 3D Occupancy Grid

Backed by a `TinyNPY` array (flat `int8_t` in row-major $X \times Y \times Z$ order). Implements both `IMap3D` (read-only) and `IMutableMap3D` (write + persist).

Voxel occupancy values follow the `VoxelOccupancy` enum:

Value	Name	Meaning
-3	PotentiallyOccupied	Seen on beam path but not confirmed solid
-2	OutOfBounds	Queried position is outside map boundaries
-1	Unmapped	Not yet scanned by the drone (initial state)
0	Empty	Scanned, confirmed open
1	Occupied	Scanned, confirmed solid (wall/obstacle)

A `MapConfig` carries the voxel resolution (in cm), world-space offset, and axis sizes. Resolution negotiation: the factory picks the smaller of the mission-requested resolution and the map's native resolution; if the request is smaller than the native resolution it is silently ignored (`ResolutionRequestStatus::Ignored`).

5.11 ConfigParser — YAML Configuration

All configuration files use YAML. Free functions parse each file type into a typed struct:

- `parseDroneConfig()` → `DroneConfigData` — physical dimensions, max move distances.
- `parseLidarConfig()` → `LidarConfigData` — `z_min`, `z_max`, step size `d`, FOVC circles.
- `parseMissionConfig()` → `MissionConfigData` — max steps, GPS resolution, output resolution factor, optional bounds.
- `parseSimulationConfig()` → `SimulationConfigData` — map file, resolution, offset, initial drone pose.
- `parseCompositionConfig()` → `SimulationCompositionData` — all of the above via file path lists.
- `parseComparisonConfig()` → `ComparisonConfigData` — used by `maps_comparison` tool.

Errors are appended to a `std::string&` output parameter and never throw; callers decide whether to abort or continue with defaults.

5.12 ErrorLogger

Writes structured error entries (code + message) to a log file on disk. Created by `SimulationRunFactory` when config parsing produces non-fatal errors; the log path is included in the output YAML report.

5.13 MapsComparison — Scoring

Computes an F1-based accuracy score (0–100) between a ground-truth map and one or more drone output maps. For occupied voxels it computes recall (true positives / ground-truth positives) and precision (true positives / drone positives), then combines them as $F1 = 2 \cdot \text{recall} \cdot \text{precision} / (\text{recall} + \text{precision})$. This penalises both missed obstacles (low recall) and false-positive obstacles (low precision).

5.14 ScanResultToVoxels

Converts a `LidarScanResult` (a set of beam hits and misses expressed as world-space distances) into voxel writes on the `IMutableMap3D` output map. For each beam: marches along the beam direction marking cells `Empty` until the reported distance; marks the hit cell `Occupied`. Occupied cells are never overwritten with `Empty`.

5.15 Types — Strong-Typed Units (mp-units)

All physical quantities use the **mp-units** library (v2.5.0, via `vcpkg`) to enforce unit safety at compile time with zero runtime overhead.

```
using PhysicalLength = mp_units::quantity<drone_mapper::cm, double>;
using HorizontalAngle = mp_units::quantity<mp_units::angular::degree, double>;
```

- **Construction:** `5.0 * cm` → `PhysicalLength`, `90.0 * deg` → `HorizontalAngle`
- **Extraction:** `qty.numerical_value_in(cm)` / `qty.numerical_value_in(deg)` → `double`

Custom quantity specs `x_extent`, `y_extent`, `z_extent` are defined as subspecifications of `isq::length` to keep axis-labelled quantities type-safe.

6. File I/O Summary

Direction	File / Pattern	Format	Description
Input	sim_compose.yaml	YAML	Lists all config files for a batch run (nested simulation_mission_groups)
Input	simulation/*.yaml	YAML	Map path, resolution, drone start pose (map-local), map_axes_offset (shifts npy origin to world coords)
Input	missions/mission*.yaml	YAML	max_steps, GPS resolution, output factor, optional bounds
Input	drone_configs/drone*.yaml	YAML	Physical drone dimensions, max movement
Input	lidar_configs/lidar*.yaml	YAML	z_min, z_max, step size d, FOVC circles
Input	*.npy	NumPy float32 3D array	Ground-truth voxel occupancy grid
Output	output_results/<map>_ms<N>_d<R>_fov<F>/output_map.npy	NumPy int8	Drone's reconstructed occupancy map
Output	simulation_output.yaml	YAML	Summary report: scores, statuses, file paths for all runs
Output	output_results/errors_<run>.log	Plain text	Created only when recoverable parse errors occurred

7. Design Considerations and Alternatives

7.1 Dependency Inversion Throughout

`DroneControlImpl` depends only on five abstract interfaces; `MissionControlImpl` depends only on `IDroneControl`; `MappingAlgorithmImpl` depends only on `IMap3D`. All concrete objects (mocks, map, algorithm) are created in `SimulationRunFactory` and injected. This makes every layer independently testable and decouples algorithm logic from simulation infrastructure.

Alternative considered: direct instantiation inside `SimulationRunImpl`. Rejected because it prevents unit-testing individual layers and couples algorithm logic to mock implementations.

7.2 Factory Pattern for Combinatorial Runs

`SimulationRunFactory` encapsulates all object construction and wiring for a single run. The `SimulationManager` only iterates the combination space and delegates creation. This cleanly separates "what to run" from "how to build it".

Alternative considered: building everything inline in `SimulationManager`. Rejected because it makes `SimulationManager` impossible to test without real files and real objects.

7.3 NPY as Map Format

Ground-truth and output maps are stored as NumPy `.npy` float32 arrays (via TinyNPY). This format is compact, binary, trivially readable from Python for analysis, and avoids parsing overhead at load time.

Alternative considered: CSV text files (as in Assignment 1). Rejected because they are slower to parse, larger on disk, and require custom readers in analysis scripts.

7.4 Deterministic DFS vs. Other Exploration Strategies

Fixed neighbour-search order (+X, -X, +Y, -Y, +Z, -Z) makes exploration fully deterministic and reproducible.

Alternatives considered:

- **BFS:** Layer-by-layer coverage, but higher memory (frontier queue) and longer backtrack paths.
- **Frontier-based exploration:** Prioritises boundary cells; more efficient for sparse maps but significantly more complex and harder to guarantee determinism.
- **Random walk:** Simple but non-deterministic with no coverage guarantee.

7.5 F1 Scoring vs. Recall-Only

The scoring metric uses $F1 = 2 \cdot \text{recall} \cdot \text{precision} / (\text{recall} + \text{precision})$ for occupied cells. Pure recall rewarded mapping every voxel as occupied (trivially 100%); F1 penalises false positives equally to missed voxels.

Alternative considered: recall only. Rejected because it does not penalise false-positive obstacle detections, allowing trivial cheating by marking everything occupied.

7.6 Strong Types with mp-units

All physical quantities use **mp-units** for compile-time unit safety — prevents unit-mismatch bugs with zero runtime overhead. Axis-specific quantity specs (`x_extent` , `y_extent` , `z_extent`) further prevent accidental axis confusion at compile time.

Alternative considered: raw `double` with naming conventions. Rejected because the compiler cannot enforce unit correctness and unit bugs are historically hard to find (e.g. Mars Climate Orbiter).

7.7 YAML Configuration vs. Key=Value Text

All configuration files use YAML (parsed via `yaml-cpp`). YAML supports nested structures, lists, and comments natively, matching the hierarchical nature of simulation composition data.

Alternative considered: flat key=value text (as in Assignment 1). Rejected because lists of config file paths cannot be expressed cleanly without special syntax, and YAML is better supported by tooling.

8. Testing Approach

8.1 Framework & Build

The project uses **Google Test (GTest) v1.17.0**, installed via **vcpkg** (along with mp-units, yaml-cpp, and TinyNPY). All tests are compiled into a single binary `drone_mapper_simulation_test` and registered with CTest. **GCC 12** (`g++-12`) is required; GCC 11 cannot compile the project due to a known compiler bug with deduction guides inside class templates triggered by the mp-units + fmt v12 combination.

8.2 Test Organisation

The test suite is organised into eight suites across three directories:

Suite	Source File	Tests	Focus
SimulationManager	<code>components/test_simulation_manager.cpp</code>	9	Null factory rejection, report structure, combinatorial run count, all drone/lidar combos run, factory exception handling, metric field
SimulationRun	<code>components/test_simulation_run.cpp</code>	14	Delegates to mission control, saves output map, result contains file path, config data propagated, score range, error missions, MockGPS position/heading/resolution, rotate changes heading, score ≤ 100 , file non-empty
MissionControl	<code>components/test_mission_control.cpp</code>	9	Completes on Done, multi-step runs, max_steps enforcement, zero max_steps, exact step count, error propagation
DroneControl	<code>components/test_drone_control.cpp</code>	14	State reflects GPS, step index, Completed/Continue results, scan called when requested, movement commands dispatched (advance/rotate/elevate), scan result passed to next step, null scan on first call, GPS state after move
MappingAlgorithm	<code>components/test_mapping_algorithm.cpp</code>	14	First call returns Working, returns Finished on empty map, occupied voxel not navigated into, no crash on full map, valid enum values, reads map state, movement limits respected (advance/elevate/rotate), always requests scan

MockLidar	components/test_mock_lidar.cpp	17	Empty map returns max-distance, wall detection, distance bounded by z_max, FOVC beam counts (0/1/2/3/4 circles), directional accuracy, z_min behaviour, exact distance, obstacle at z_max boundary, config() returns configured data, hit angle matches scan orientation, closest obstacle detected first
MapsComparison	components/test_maps_comparison.cpp	12	Identical empty maps, empty drone vs occupied GT, perfect mapping, identical full maps = 100, missed occupied, score range 0–100, multiple targets, false positives reduce score, copied map scores 100, partial mapping, empty vs empty GT
Integration	integration/test_integration_real_algorithm.cpp integration/test_integration_mock_algorithm.cpp	25	Real algorithm no crash, mission terminates, score in range, end-to-end SimulationRunImpl, benchmark map loads/runs/scores, new scenario maps (small/big/house), sim_compose parses correctly, drone starts inside map bounds, scan populates output map, lidar hits marked Occupied, mission step count in bounds

Total: 114 automated tests, all passing.

8.3 Testing Strategy

- **Unit tests (components/):** Each component is tested in isolation with mock dependencies injected via constructor. Components are tested against controlled synthetic maps and deterministic mock behaviours.
- **Integration tests (integration/):** Wire the full chain (SimulationRunImpl + all mocks + real algorithm + Map3DImpl) over small synthetic maps, verifying end-to-end exploration behaviour, termination, and scoring.
- **Dependency injection enables isolation:** Because every layer depends only on abstract interfaces, each component can be tested independently without instantiating the full object graph.
- **Shared test helpers:** Factory functions (`makeLidarConfig()` , `makeSimpleMap()`) and mock implementations (mock algorithm, mock GPS) reduce boilerplate across suites.

8.4 How to Run

```
export VCPKG_ROOT=/path/to/vcpkg
cmake --preset default
cmake --build build
./build/drone_mapper_simulation_test          # 75 tests
./build/drone_mapper_simulation sample_config/simulation_compositions.yaml
./build/maps_comparison data_maps/office_small.npy data_maps/office_small.npy
```

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . --config Release
ctest --output-on-failure
```

9. Glossary

Term	Definition
FOVC	Field Of View Circles — number of beam rings in the Lidar cone (including the centre beam).
Grid / Voxel	A 3D integer-index cell produced by dividing continuous world coordinates by the map resolution.
Ground Truth / Hidden Map	The reference building map loaded from a <code>.npy</code> file; never directly accessible to the algorithm.
z_min / z_max	Minimum / maximum measurable Lidar range (cm).
d	Lidar step size — angular spacing between beam rings (cm at z_min).
Composition	A YAML file listing simulation, mission, drone, and lidar config files; the Cartesian product defines all runs.
F1 Score	Harmonic mean of recall and precision; used to score occupied-voxel reconstruction accuracy.
DIP	Dependency Inversion Principle — high-level modules depend on abstractions, not on concretions.